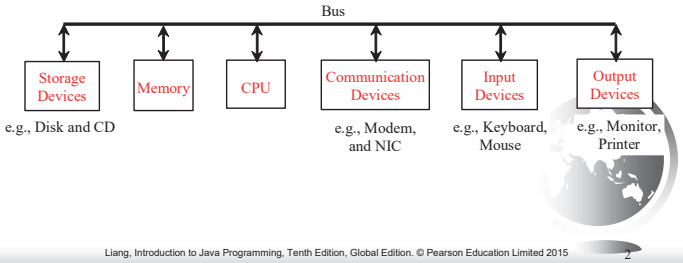
Chapter 1 Introduction to Computers, Programs, and Java

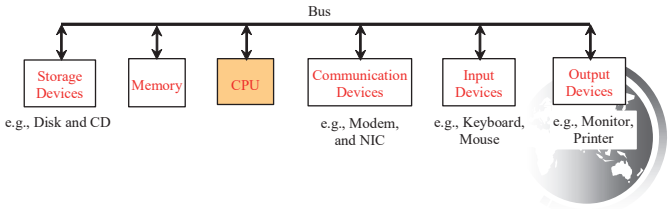
What is a Computer?

- A computer is an electronic device that stores and processes data
- A computer includes both *hardware* and *software.*
- A computer consists of a CPU, memory, hard disk, floppy disk, monitor, printer, and communication devices.



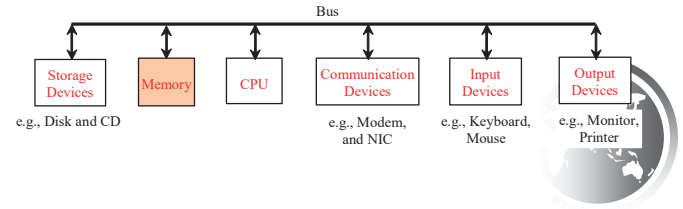
CPU

- The central processing unit (CPU) is the brain of a computer. It retrieves instructions from memory and executes them.
- The CPU speed is measured in megahertz (MHz), with 1 megahertz equaling 1 million pulses per second. The speed of the CPU has been improved continuously. Intel's newest processors run at 3 gigahertz (1 gigahertz is 1000 megahertz).



Memory

Memory is to store data and program instructions for CPU to execute. A memory unit is an ordered sequence of bytes, each holds eight bits. A program and its data must be brought to memory before they can be executed. A memory byte is never empty, but its initial content may be meaningless to your program. The current content of a memory byte is lost whenever new information is placed in it.



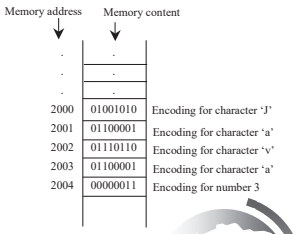
How Data is Stored?

Data of various kinds, such as numbers, characters, and strings, are encoded as a series of bits (zeros and ones).

Computers use zeros and ones because digital devices have two stable states, which are referred to as *zero* and *one* by convention.

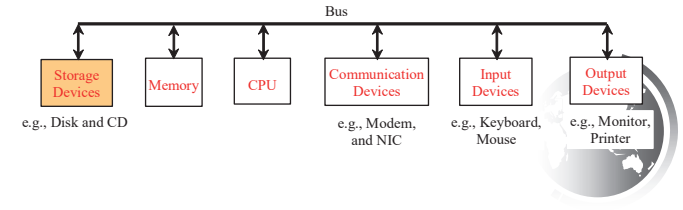
The programmers need not to be concerned about the encoding and decoding of data, which is performed automatically by the system based on the encoding scheme.

For example, character 'J' is represented by 01001010 in one byte. No two data can share or split a same byte. A byte is the minimum storage unit.



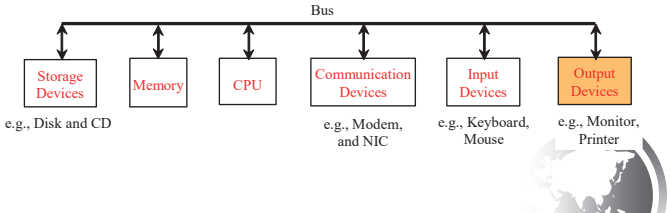
Storage Devices

Memory is volatile, because information is lost when the power is off. Programs and data are permanently stored on storage devices and are moved to memory when the computer actually uses them. There are three main types of storage devices: Disk drives (hard disks and floppy disks), CD drives (CD-R and CD-RW), and USB flash drives.



Output Devices: Monitor

The monitor displays information (text and graphics). The resolution and dot pitch determine the quality of the display.



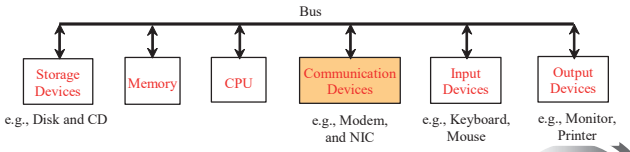
Monitor Resolution and Dot Pitch

resolution The *screen resolution* specifies the number of pixels in horizontal and vertical dimensions of the display device. *Pixels* (short for "picture elements") are tiny dots that form an image on the screen. A common resolution for a 17-inch screen, for example, is 1,024 pixels wide and 768 pixels high. The resolution can be set manually. The higher the resolution, the sharper and clearer the image is.

dot pitch The *dot pitch* is the amount of space between pixels, measured in millimeters. The smaller the dot pitch, the sharper the display.

Communication Devices

Computers can be networked through communication devices, such as a *dial-up modem,* a *DSL* (digital subscriber line) or *cable modem*, a wired Network interface card (*NIC*) or a *wireless adapter*.



Programs

- Computer *programs*, known as *software*, are instructions to the computer.
- You tell a computer what to do through programs. Without programs, a computer is an empty machine.
- Computers do not understand human languages, so you need to use computer languages to communicate with them.
- Programs are written using programming languages.

Programming Languages

Machine Language Assembly Language High-Level Language

Machine language is a set of primitive instructions built into every computer.

The instructions are in the form of binary code, so you have to enter binary codes for various instructions.

For example, to add two numbers, you might write an instruction in binary like this:

```
1101101010011010
```

Programming in machine language is a tedious process. Moreover, programs written in machine language are very difficult to read and modify.

Programming Languages

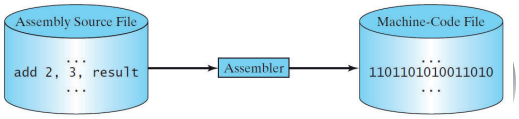
Machine Language Assembly Language High-Level Language

Assembly languages were developed to make programming easy.

The computer cannot understand assembly language, however, a program called assembler is used to convert assembly language programs into machine code.

For example, to add two numbers, you might write an instruction in assembly code like this:

```
ADDF3 R1, R2, R3
```



Programming Languages

Machine Language Assembly Language High-Level Language

The high-level languages are English-like and easy to learn and program.

The instructions in a high level languages are called *statements*.

For example, the following is a high-level language statement that computes the area of a circle with radius 5:

```
area = 5 * 5 * 3.1415;
```


Popular High-Level Languages

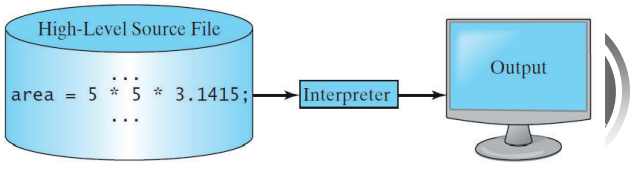
Language	Description
Ada	Named for Ada Lovelace, who worked on mechanical general-purpose computers. The Ada language was developed for the Department of Defense and is used mainly in defense projects.
BASIC	Beginner's All-purpose Symbolic Instruction Code. It was designed to be learned and used easily by beginners.
C	Developed at Bell Laboratories. C combines the power of an assembly language with the ease of use and portability of a high-level language.
C++	C++ is an object-oriented language, based on C.
C#	Pronounced "C Sharp." It is a hybrid of Java and C++ and was developed by Microsoft.
COBOL	COmmon Business Oriented Language. Used for business applications.
FORTRAN	FORmula TRANslation. Popular for scientific and mathematical applications.
Java	Developed by Sun Microsystems, now part of Oracle. It is widely used for developing platform-independent Internet applications.
Pascal	Named for Blaise Pascal, who pioneered calculating machines in the seventeenth century. It is a simple, structured, general-purpose language primarily for teaching programming.
Python	A simple general-purpose scripting language good for writing short programs.
Visual Basic	Visual Basic was developed by Microsoft and it enables the programmers to rapidly develop graphical user interfaces.

Interpreting/Compiling Source Code

- A program written in a high-level language is called a *source program* or *source code*.
- Because a computer cannot understand a source program, a source program must be translated into machine code for execution.
- The translation can be done using another programming tool called an *interpreter* or a *compiler*.

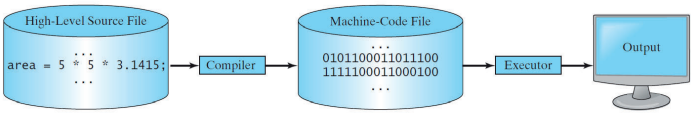
Interpreting Source Code

- An interpreter reads one statement from the source code, translates it to the machine code or virtual machine code, and then executes it right away, as shown in the following figure.
- Note that a statement from the source code may be translated into several machine instructions.



Compiling Source Code

A compiler translates the entire source code into a machine-code file, and the machine-code file is then executed, as shown in the following figure.

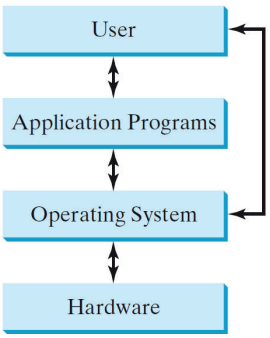


Operating Systems

The *operating system* (OS) is a program that manages and controls a computer's activities.

The popular operating systems for general-purpose computers are Microsoft Windows, Mac OS, and Linux.

Application programs, such as a Web browser or a word processor, cannot run unless an operating system is installed and running on the computer.



Why Java?

The answer is that Java enables users to develop and deploy applications on the Internet for servers, desktop computers, and small hand-held devices.

The future of computing is being profoundly influenced by the Internet, and Java promises to remain a big part of that future. Java is the Internet programming language.

- Java is a general purpose programming language.
- Java is the Internet programming language.

Java, Web, and Beyond

- Java can be used to develop
  - standalone applications.
  - applications running from a browser.
  - applications for hand-held devices.
  - applications for Web servers.

The Java Language Specification, API, JDK, and IDE

- The *Java language specification* is a technical definition of the Java programming language's syntax and semantics.
- The *application program interface (API)*, also known as *library*, contains predefined classes and interfaces for developing Java programs.
- The JDK (Java Development Kit) is the software for developing and running Java programs.

JDK Editions

- Java Standard Edition (J2SE)
  - J2SE can be used to develop client-side standalone applications or applets.
- Java Enterprise Edition (J2EE)
  - J2EE can be used to develop server-side applications such as Java servlets, Java ServerPages, and Java ServerFaces.
- Java Micro Edition (J2ME).
  - J2ME can be used to develop applications for mobile devices such as cell phones.

This book uses J2SE to introduce Java programming.

Popular Java Integrated Development Environments (IDEs)

An IDE is an integrated development environment for rapidly developing programs.

- NetBeans
- Eclipse

A Simple Java Program

Listing 1.1 Welcome.java

```
// This program prints Welcome to Java!

public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Program Structure

```
class CLASSNAME {
 public static void main(String[] args){
    STATEMENTS;
  }
}
```


animation

Trace a Program Execution

Enter main method

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


animation

Trace a Program Execution

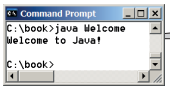
Execute statement

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


animation

Trace a Program Execution

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```



print a message to the console

Anatomy of a Java Program

- Class name
- Main method
- Statements
- Statement terminator
- Reserved words
- Comments
- Blocks

Class Name

- Every Java program must have at least one class.
- Each class has a name.
- By convention, class names start with an uppercase letter.
  - In this example, the class name is Welcome.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


Main Method

- In order to run a class, the class must contain a method named ***main***.
- The program is executed from the main method.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


Statement

- A statement represents an action or a sequence of actions.
- The statement System.out.println("Welcome to Java!") in the program in Listing 1.1 is a statement to display the greeting "Welcome to Java! ".

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Statement Terminator

Every statement in Java ends with a semicolon (;).

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


Reserved words

- Reserved words or keywords are words that have a specific meaning to the compiler and cannot be used for other purposes in the program.
- For example, when the compiler sees the word *class*, it understands that the word after *class* is the name for the class.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


Comments

- Comments help programmers to communicate and understand the program.
- They are not programming statements and thus are ignored by the compiler.
- Comments are preceded by two slashes (//) on a line, called a line comment, or enclosed between /* and */ on one or several lines, called a block comment or paragraph comment.

```
// This program prints Welcome to Java!
/* This program prints Welcome to Java! */
/* This program prints
  Welcome to Java! */
```


Blocks

- A pair of braces in a program forms a block that groups components of a program.
- Each block begins with an opening brace ({) and ends with a closing brace (}).

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Class block

Method block

Caution !!!

- Java source programs are case sensitive.
- It would be wrong, for example, to replace **main** in the program with **Main**.

Special Symbols

Character	Name	Description
{}	Opening and closing braces	Denotes a block to enclose statements.
()	Opening and closing parentheses	Used with methods.
[]	Opening and closing brackets	Denotes an array.
//	Double slashes	Precedes a comment line.
" "	Opening and closing quotation marks	Enclosing a string (i.e., sequence of characters).
;	Semicolon	Marks the end of a statement.

Opening and closing braces { … }

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


Opening and closing parentheses (…)

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Semicolon

;

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


Double slashes

// ...

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


Opening and closing quotation marks

" ... "

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```


Example 1. WelcomeWithThreeMessages.java

```
public class WelcomeWithThreeMessages {
  public static void main(String[] args) {
    System.out.println("Programming is fun!");
    System.out.println("Fundamentals First");
    System.out.println("Problem Driven");
  }
}
```

Output:

```
Programming is fun!
Fundamentals First
Problem Driven
```


Example 2. ComputeExpression.java

example of evaluating $\frac{10.5 + 2 \times 3}{45 - 3.5}$

```
public class ComputeExpression {
  public static void main(String[] args) {
    System.out.println((10.5 + 2 * 3) / (45 - 3.5));
  }
}
```

Output:

```
0.39759036144578314
```

The multiplication operator in java is *.

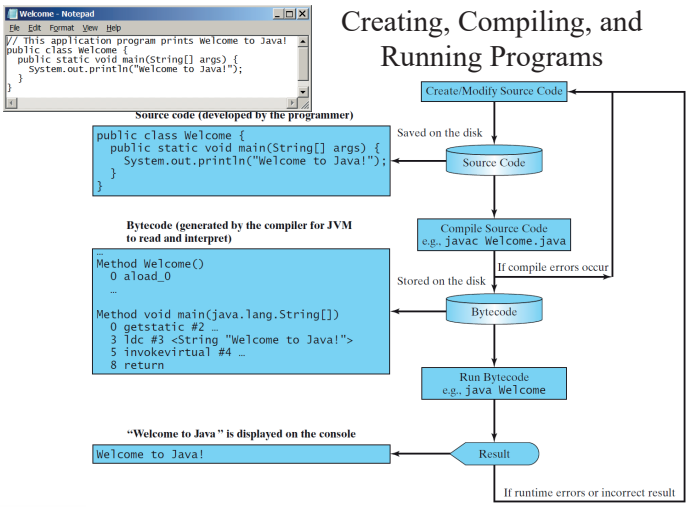
Example 3.

```
public class Test {
    public static void main(String[] args) {
        System.out.println("3.5 * 4 / 2 - 2.5 is");
        System.out.println(3.5 * 4 / 2 - 2.5);
    }
}
```

Output:

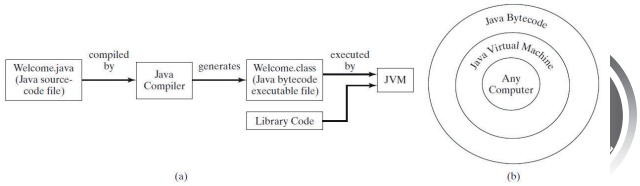
```
3.5 * 4 / 2 - 2.5 is
4.5
```


Creating, Compiling, and Running Programs



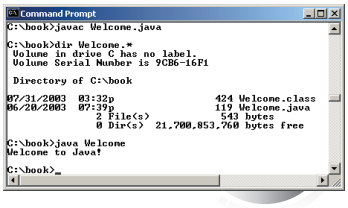
Compiling Java Source Code

- Java was designed to run object programs on any platform.
- With Java, you write the program once, and compile the source program (Welcome.java) into a special type of object code, known as *bytecode* (Welcome.class).
- The bytecode can then run on any computer with a Java Virtual Machine, as shown below. Java Virtual Machine is a software that interprets Java bytecode.



Compiling and Running Java from the Command Window

- Compile
  - javac Welcome.java
- Run
  - java Welcome



Programming Style and Documentation

Good programming style and proper documentation make a program easy to read and help programmers prevent errors.

- Appropriate Comments
- Naming Conventions
- Proper Indentation and Spacing Lines
- Block Styles

Appropriate Comments

- Include a summary at the beginning of the program to explain what the program does, its key features, its supporting data structures, and any unique techniques it uses.
- Include your name, class section, instructor, date, and a brief description at the beginning of the program.

Naming Conventions

- Choose meaningful and descriptive names.
- Class names:
  - Capitalize the first letter of each word in the name. For example, the class name `ComputeExpression`.

Proper Indentation and Spacing

- Indentation
  - Indent two spaces.
- Spacing
  - Use blank line to separate segments of the code.

```
System.out.println(3+4*4); // Bad style
System.out.println(3 + 4 * 4); // Good style
```


Example: Reformat the following program

```
public class Test
{
// Main method
public static void main(String[] args) {
/** Display output */
System.out.println("Welcome to Java");
}
}
```

```
public class Test{
  // Main method
  public static void main(String[] args) {
    /** Display output */
    System.out.println("Welcome to Java!");
  }
}
```


Block Styles

Use end-of-line style for braces.

Next-line style

```
public class Test
{
  public static void main(String[] args)
  {
    System.out.println("Block Styles");
  }
}
```

End-of-line style

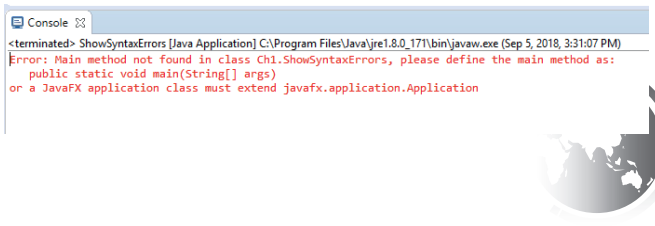
```
public class Test {
  public static void main(String[] args) {
    System.out.println("Block Styles");
  }
}
```


Programming Errors

- Syntax Errors
  - Detected by the compiler
- Runtime Errors
  - Causes the program to abort
- Logic Errors
  - Produces incorrect result

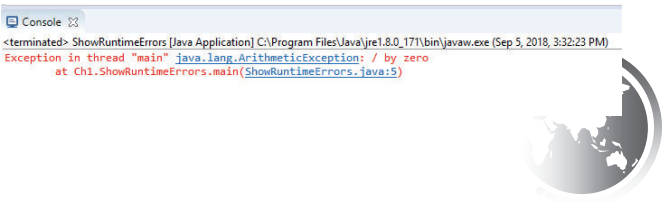
Syntax Errors

```
public class ShowSyntaxErrors {
  public static main(String[] args) {
    System.out.println("Welcome to Java);
  }
}
```



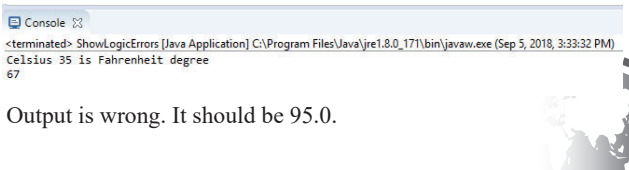
Runtime Errors

```
public class ShowRuntimeErrors {
  public static void main(String[] args) {
    System.out.println(1 / 0);
  }
}
```



Logic Errors

```
public class ShowLogicErrors {
  public static void main(String[] args) {
    System.out.println("Celsius 35 is Fahrenheit degree ");
    System.out.println((9 / 5) * 35 + 32);
  }
}
```



Output is wrong. It should be 95.0.

Common Errors

- Missing a closing brace
- Missing a semicolon
- Missing quotation marks for strings
- Misspelling names

Common Error 1: Missing Braces

```
public class Welcome {
}
```

Type this closing brace right away to match the opening brace

Common Error 2: Missing Semicolons

```
public static void main(String[] args) {
  System.out.println("Programming is fun!");
  System.out.println("Fundamentals First");
  System.out.println("Problem Driven")
}
```

Missing semicolon

Common Error 3: Missing Quotation Marks

```
public static void main(String[] args) {
  System.out.println("Programming is fun! );
}
```

Missing a quotation mark

Common Error 4: Misspelling Names

```
public static void Main(string[] args) {
  System.out.println("Programming is fun! ");
}
```

main is misspelled as **Main**

String is misspelled as **string**